

PERSONAL PORTFOLIO BUILDER USING AWS

*Prepared in the partial fulfillment of the Summer Internship Program on
AWS*

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Anil Kumar Varanasi, APSSDC

Submitted by

*SESETTI VENKAT VASANTH,5241411196,GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE & PG COURSE
(AUTONOMOUS),(BATCH-139)*

THERUKUTI PRANAY SREEKAR,25985A4612,RAGHU ENGINEERING COLLEGE,(BATCH-139)

JATLA ANUSHA,25NM5A4306,VIGNAN'S INSTITUTE OF ENGINEERING FOR WOMEN,(BATCH-139)

*KALISSETTI BHAGYA SRI,323114110065,ANDHRA UNIVERSITY COLLEGE OF ENGINEERING FOR
WOMEN,(BATCH-139)*

*MOGILI GOWTHAMI,323114110068,ANDHRA UNIVERSITY COLLEGE OF ENGINEERING FOR
WOMEN,(BATCH-139)*

*DAMACHARLA MANUJA LATHA,5241421020,GAYATRI VIDYA PARISHAD COLLEGE FOR PG & PG COURSES
(AUTONOMOUS),(BATCH-139)*

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Anil Kumar Varanasi, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Sesetti Venkat Vasanth,5241411196,vasanthsessetti@gmail.com
Therukuti. Pranay sreekar, (25985A4612),pranaysreekar7@gmail.com
Jatla Anusha,25NM5A4306,22295cm019@gmail.com
Kalisetti Bhagya sri,323114110065,bhagyasrikalisetti2005@gmail.com
Mogili Gowthami,323114110068,gowthamimogili3@gmail.com
Damacharla Manuja Latha,5241421020,manujalathadamacharla@gmail.com

ABSTRACT

DevVista – Portfolio Generator is a web-based application developed to simplify the creation of professional digital portfolios. The application enables users to create a personalized portfolio by entering their personal details, educational qualifications, technical skills, projects, certifications, achievements, and contact information through a simple and user-friendly interface. The system automatically organizes the provided information into a well-structured and visually appealing portfolio website, eliminating the need for coding knowledge

The project is designed to help students, job seekers, freelancers, and professionals showcase their profiles effectively in the digital world. It provides an efficient platform for managing and updating portfolio content while ensuring a responsive design that works seamlessly across desktops, tablets, and mobile devices. Users can customize their portfolio and present their skills and accomplishments in a professional format.

DevVista significantly reduces the time and effort required to build a portfolio manually while improving the overall presentation of user profiles. The generated portfolio can be easily shared with recruiters, employers, and academic institutions through a single link, helping users strengthen their online presence and enhance career opportunities.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION	5
2.	METHODOLOGY	6
3.	TECHNOLOGY STACK	9
4.	SYSTEM DESIGN / ARCHITECTURE.....	11
5.	IMPLEMENTATION	23
6.	RESULTS	26
7.	CONCLUSION.....	29

1. INTRODUCTION

In today's digital world, having a professional online portfolio has become essential for students, developers, and job seekers. A portfolio helps individuals showcase their skills, projects, achievements, certifications, and educational background in an organized and attractive manner. However, creating a portfolio website from scratch requires technical knowledge of web development, which many students may not possess.

DevVista – Cloud Hosted Portfolio Generator is developed to solve this problem by providing a simple and user-friendly platform for creating professional portfolios. The application enables users to select a portfolio template, enter their personal and academic details, add projects and skills, preview the portfolio, and publish it online without writing any code.

The project is developed using **HTML, CSS, and JavaScript** and is hosted on **Amazon Web Services (AWS)** using **Amazon S3 Static Website Hosting**. The website is fully responsive and can be accessed from desktops, laptops, tablets, and mobile devices. AWS hosting ensures high availability, reliability, and fast access to the application.

In addition to portfolio creation, DevVista focuses on providing a smooth and interactive user experience through a clean interface and responsive design.

Users can easily navigate through different templates, customize their portfolio according to their preferences, and instantly preview the changes before publishing.

The primary objective of DevVista is to help students and professionals build an impressive online presence quickly and efficiently. The project demonstrates the practical use of modern web technologies and cloud computing to develop a real-world application that simplifies portfolio creation and improves professional visibility.

2. METHODOLOGY

The development of DevVista – Cloud Hosted Portfolio Generator followed a systematic methodology to ensure that the application is user-friendly, responsive, and efficient. The project was designed and implemented through a sequence of well-defined phases, starting from requirement analysis to deployment on AWS. Each phase contributed to building a reliable portfolio generation platform.

2.1. REQUIREMENTS GATHERING

The requirements gathering phase involved understanding the needs of students, developers, and professionals who wanted to create an online portfolio easily. Information was collected about the essential features required for the application, such as portfolio templates, user registration, portfolio customization, live preview, and online publishing. The system was designed to be simple, responsive, and easy to use without requiring coding knowledge. Additional requirements like fast loading, mobile compatibility, and secure cloud hosting using AWS were also considered. These requirements served as the foundation for designing and developing the DevVista Portfolio Generator.

2.2.DESIGN

The requirements gathering phase involved understanding the needs of students, developers, and professionals who wanted to create an online portfolio easily. Information was collected about the essential features required for the application, such as portfolio templates, user registration, portfolio customization, live preview, and online publishing. The system was designed to be simple, responsive, and easy to use without requiring coding knowledge. Additional requirements like fast loading, mobile compatibility, and secure cloud hosting using AWS were also considered. These requirements served as the foundation for designing and developing the DevVista Portfolio Generator.

2.3.IMPLEMENTATION

The DevVista Portfolio Generator was developed using HTML, CSS, and JavaScript. The application allows users to create and preview portfolios through a simple and responsive interface. Finally, the website was deployed on AWS S3 for online access.

2.4.DATABASE MANAGEMENT

The project uses a simple data management approach to organize portfolio information. User details such as personal information, skills, education, and projects are managed efficiently to generate the portfolio.

2.5.TESTING

The application was tested to ensure that all features worked correctly. Different pages and portfolio templates were checked for proper functionality, responsiveness, and user experience.

2.6 DEPLOYMENT

After successful testing, the DevVista Portfolio Generator was deployed on Amazon Web Services (AWS) using Amazon S3 Static Website Hosting. This allows users to access the application online through a web browser.

2.7 ITERATIVE REFINEMENT

The application was improved continuously by fixing errors and enhancing the user interface based on testing and feedback. These improvements helped provide a better user experience and overall performance.

3. TECHNOLOGY STACK

The **DevVista – Cloud Hosted Portfolio Generator** was developed using modern web technologies and Amazon Web Services (AWS) to build a secure, scalable, and user-friendly application. The frontend was developed using **HTML5, CSS3, and JavaScript** to provide a responsive and interactive user interface. The application was deployed on **Amazon Web Services (AWS)**, where **Amazon S3** is used for static website hosting, **Amazon API Gateway** manages communication between the client and backend services, **AWS Lambda** executes serverless backend functions, and **Amazon DynamoDB** stores user information securely. **AWS IAM** provides secure access control and permission management, while **Amazon CloudWatch** is used for monitoring application performance and maintaining execution logs. This technology stack ensures high availability, better performance, reliability, and efficient cloud-based deployment of the application.

3.1. CLOUD PLATFORM

The project is developed on Amazon Web Services (AWS), a leading cloud computing platform that provides scalable, secure, and reliable cloud services. AWS enables the application to run without managing physical servers, making deployment, maintenance, and scaling simple and cost-effective. Its serverless architecture ensures high availability and improved performance.

3.2. AWS SERVICES

The project uses multiple AWS services to build a complete serverless application. Amazon S3 hosts the frontend files, Amazon API Gateway handles client requests, AWS Lambda executes backend logic, Amazon DynamoDB stores user information, and Amazon CloudWatch monitors logs and application performance. These services work together to provide a secure and efficient system.

3.3. PROGRAMMING & SCRIPTING

The frontend is developed using HTML5, CSS3, and JavaScript to create a responsive and interactive user interface. JavaScript is also used to communicate with backend APIs and process user actions. The backend logic is implemented using Node.js in AWS Lambda to perform authentication and database operations.

3.4. DATA FORMAT

The application uses JSON (JavaScript Object Notation) as the primary data format for exchanging information between the frontend and backend services. JSON is lightweight, easy to read, and enables fast communication through REST APIs. User details and API responses are transmitted in JSON format.

3.5. SECURITY & ACCESS

Security is implemented using AWS services and best practices. API Gateway securely manages client requests, while AWS Lambda validates incoming data before processing. User passwords are

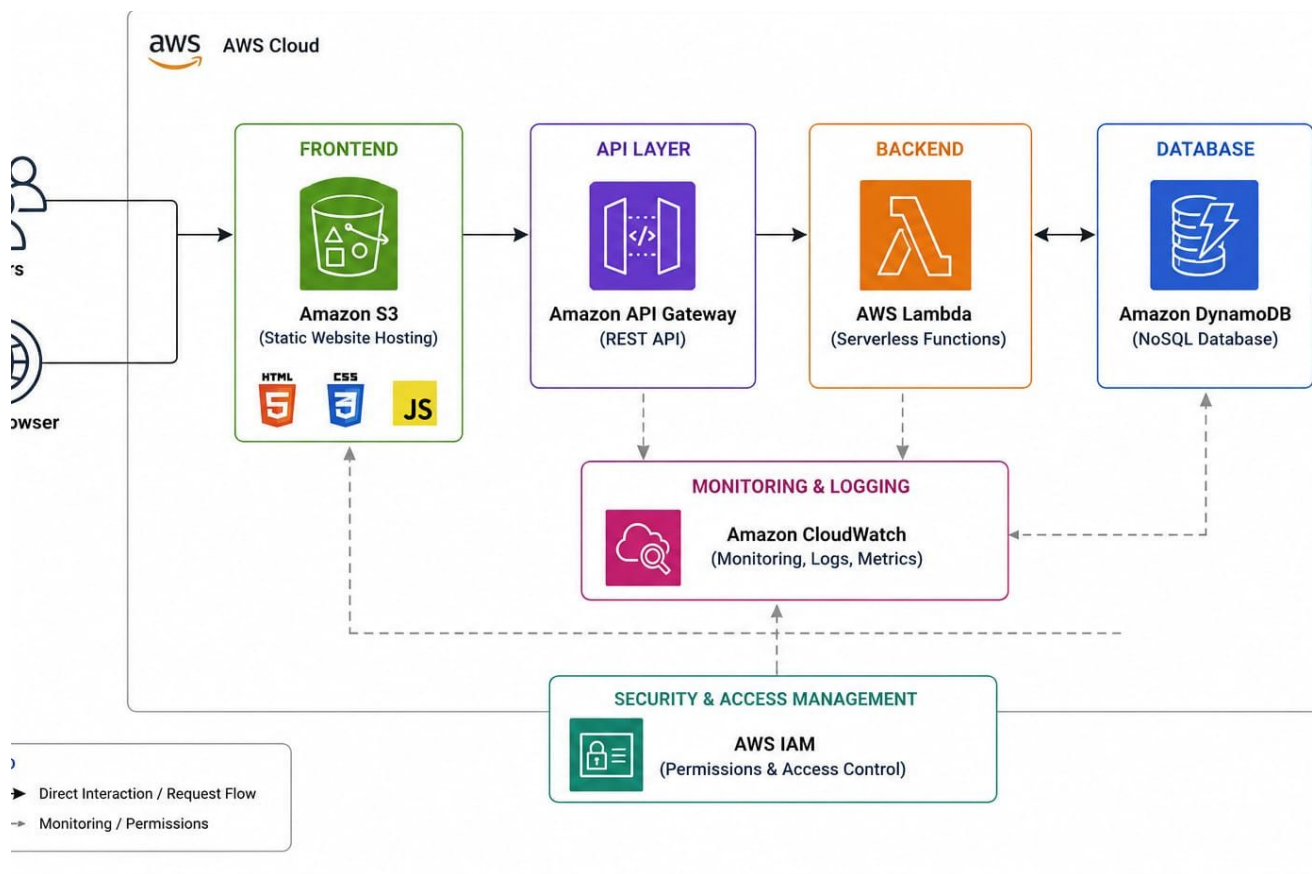
stored in encrypted form in Amazon DynamoDB, ensuring that sensitive information remains protected from unauthorized access.

3.6. MONITORING & LOGGING

Amazon CloudWatch is used to monitor the application's performance and maintain execution logs. It helps track Lambda function activities, identify errors, and support debugging to ensure the application runs smoothly.

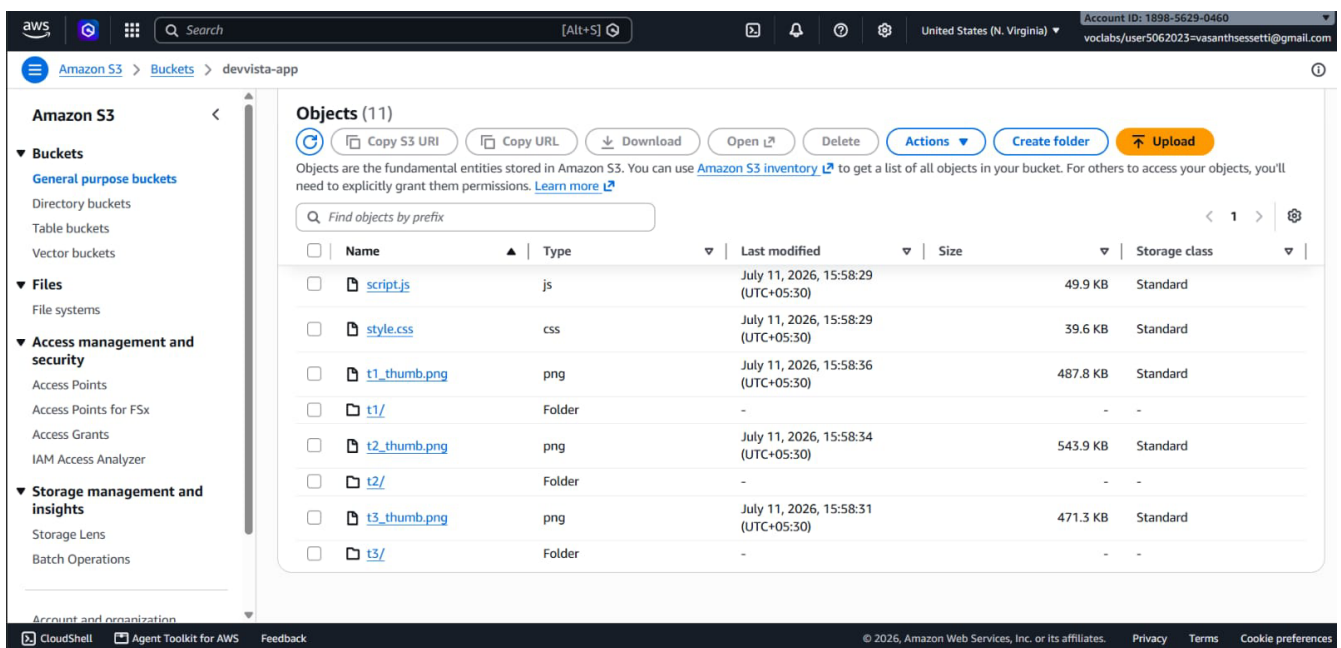
4. SYSTEM DESIGN / ARCHITECTURE

The DevVista Portfolio Generator is designed using a serverless cloud architecture on Amazon Web Services (AWS). The frontend is hosted on Amazon S3, allowing users to access the application through a web browser. User requests, such as registration and login, are sent to Amazon API Gateway, which forwards them to AWS Lambda for processing. Lambda executes the application logic, validates user input, and securely stores or retrieves data from Amazon DynamoDB.



4.1 STATIC WEBSITE HOSTING

AWS Lambda acts as the processing layer of the application. It executes backend logic such as user registration, login authentication, password encryption, and request validation. Since Lambda is serverless, it automatically scales according to incoming requests while reducing infrastructure management.



Amazon S3-frontend
hosting

The DevVista frontend files such as HTML, CSS, JavaScript, images, and portfolio templates are stored in an Amazon S3 bucket. S3 Static Website Hosting delivers the website efficiently to users with high availability and low latency.

KEY FEATURES

- * Static Website Hosting
- * Easy Integration
- * Cost-Effective Storage

4.2 MONITORING AND LOGGING

Amazon CloudWatch is a monitoring and logging service provided by AWS that helps track the performance and health of cloud applications. In the DevVista Portfolio Generator, CloudWatch collects logs generated by AWS Lambda functions and monitors application activities. It enables developers to identify errors, analyze performance, troubleshoot issues, and ensure the application runs reliably.

CloudWatch also provides real-time monitoring and supports alerts to improve the overall availability and performance of the system.

The screenshot displays the AWS CloudWatch console interface. The top navigation bar shows the AWS logo, a search bar, and account information (United States (N. Virginia), Account ID: 1898-5629-0460). The left sidebar contains the CloudWatch logo and a navigation menu with options like Favorites and recents, Ingestion, Dashboards, Alarms, AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, Logs, Metrics, and Network Monitoring. The main content area is titled 'Log Groups | CloudWatch' and shows the configuration for the log group '/aws/lambda/devvista-auth'. It includes settings for Retention (Never expire), Stored bytes (2.99 KB), Metric filters (0), Data protection (Off), and Sensitive data count (Off). Below these settings, there are tabs for Log streams, Tags, Data protection, Anomaly detection, Metric filters, Subscription filters, Contributor Insights, and Field. The 'Log streams' tab is active, showing a list of log streams with columns for Log stream, Last event time, and a checkbox for 'Show expired'. The list contains four log streams, each with a unique ID and a timestamp.

Log stream	Last event time
2026/07/14/[LATEST]711ba76abd064455a52fb0ff2d29acad	2026-07-14 15:32:35 (UTC)
2026/07/14/[LATEST]3846c8babce04faf8abece908d541b4c	2026-07-14 15:05:00 (UTC)
2026/07/11/[LATEST]ff6b31c10a05458098db5b1f913cd134	2026-07-11 13:18:45 (UTC)
2026/07/11/[LATEST]cdf9971958474f43a2460e35007f3ea9	2026-07-11 11:19:05 (UTC)

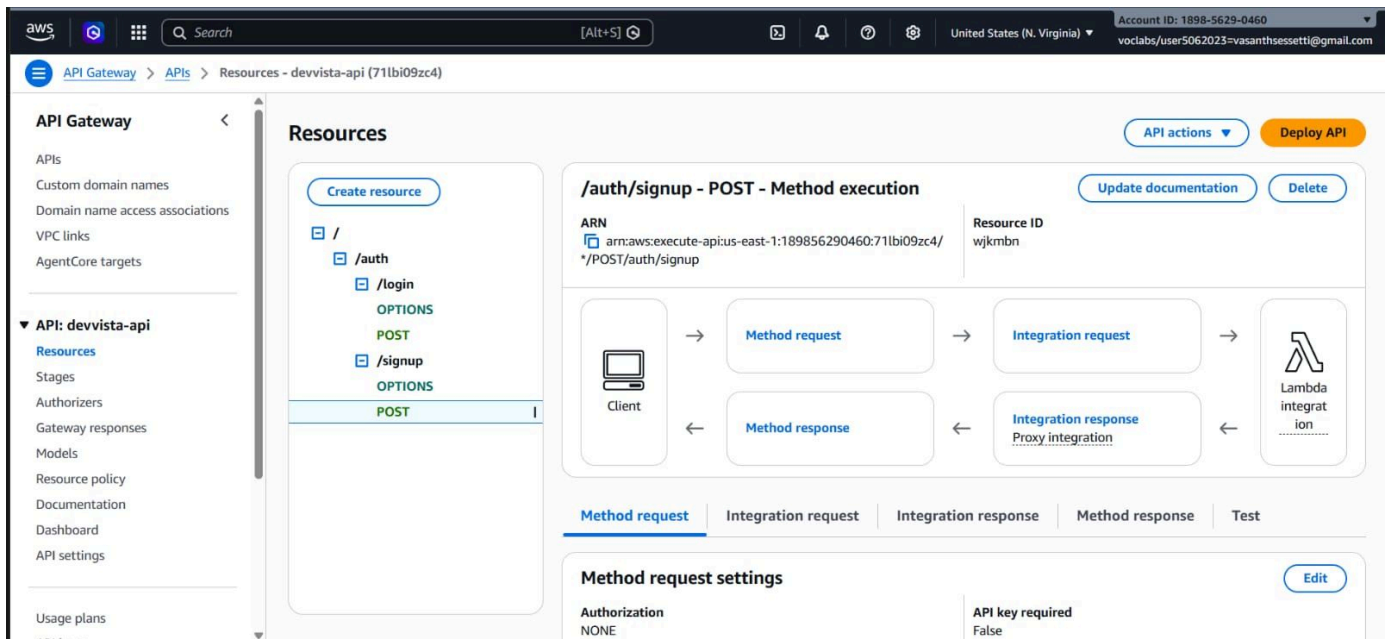
4.3 AMAZON DYNAMODB – DATA STORAGE

Amazon DynamoDB is a fully managed NoSQL database service used to store and manage the application data of the DevVista Portfolio Generator. It securely stores user registration details, login credentials, and portfolio information with high performance and low latency. DynamoDB automatically scales according to the application's workload, ensuring consistent performance even when the number of users increases. It provides built-in security, high availability, and data durability, making it a reliable database solution for cloud-based applications. The seamless integration of DynamoDB with AWS Lambda enables fast data retrieval and efficient processing of user requests, ensuring a smooth and responsive user experience.

The screenshot displays the AWS DynamoDB console interface. On the left, a navigation menu includes options like Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The main area shows the 'devvista-users' table selected. A search bar at the top of the table view contains 'Find tables'. Below it, a button labeled 'devvista-users' is visible. The table view shows a summary: 'Table: devvista-users - Items returned (3)'. A green status bar indicates 'Completed · Items returned: 3 · Items scanned: 3 · Efficiency: 100% · RCUs consumed: 2'. Below this, a table lists the returned items with columns for email, createdAt, name, and passwordHash. The footer of the console shows 'CloudShell', 'Agent Toolkit for AWS', 'Feedback', and copyright information for Amazon Web Services, Inc.

	email (String)	createdAt	name	passwordHash
☐	hdhd@gmail.com	2026-07-1...	hdhd	\$2b\$10\$f/.kc5XgpWD6Bp/1U5Sle..CzT15z...
☐	manujalathadamacha...	2026-07-1...	manuja latha	\$2b\$10\$op.8yhsDvEOyOyYic6XEkUElgsHsz...
☐	vasanthsetti@gmail...	2026-07-1...	vasanth	\$2b\$10\$XYc.CmMKJiz/lkPmntUF7.Q7c7ZZ...

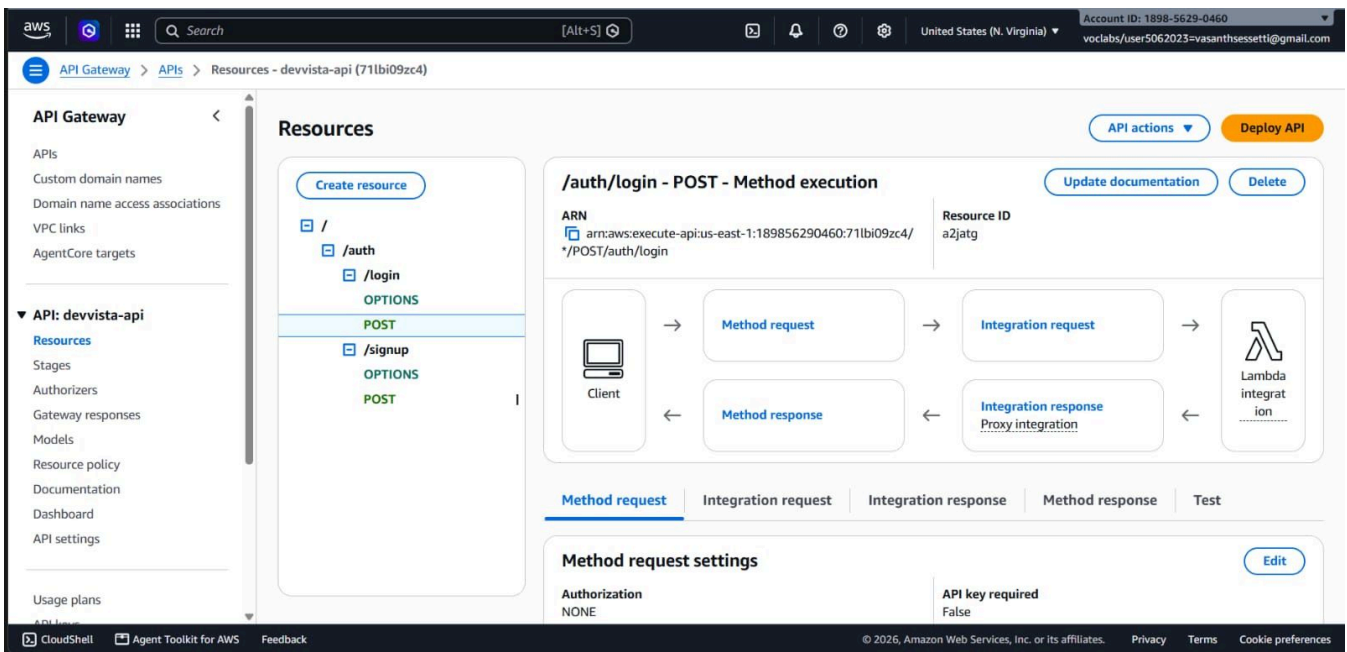
4.4. USER SIGNUP API



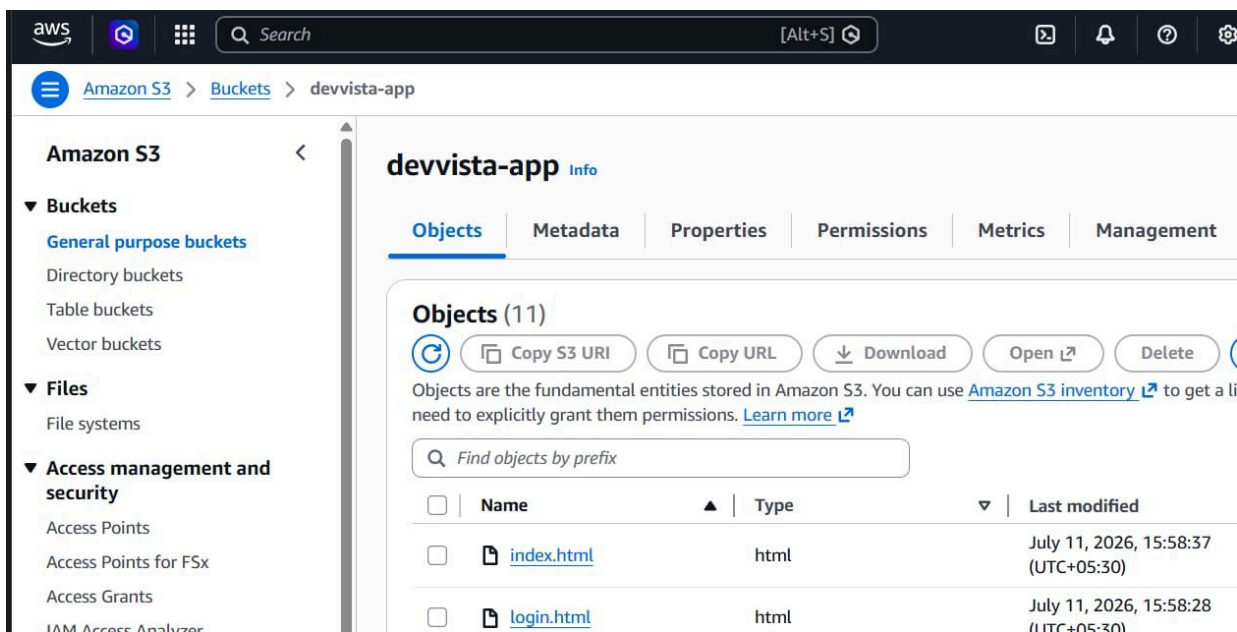
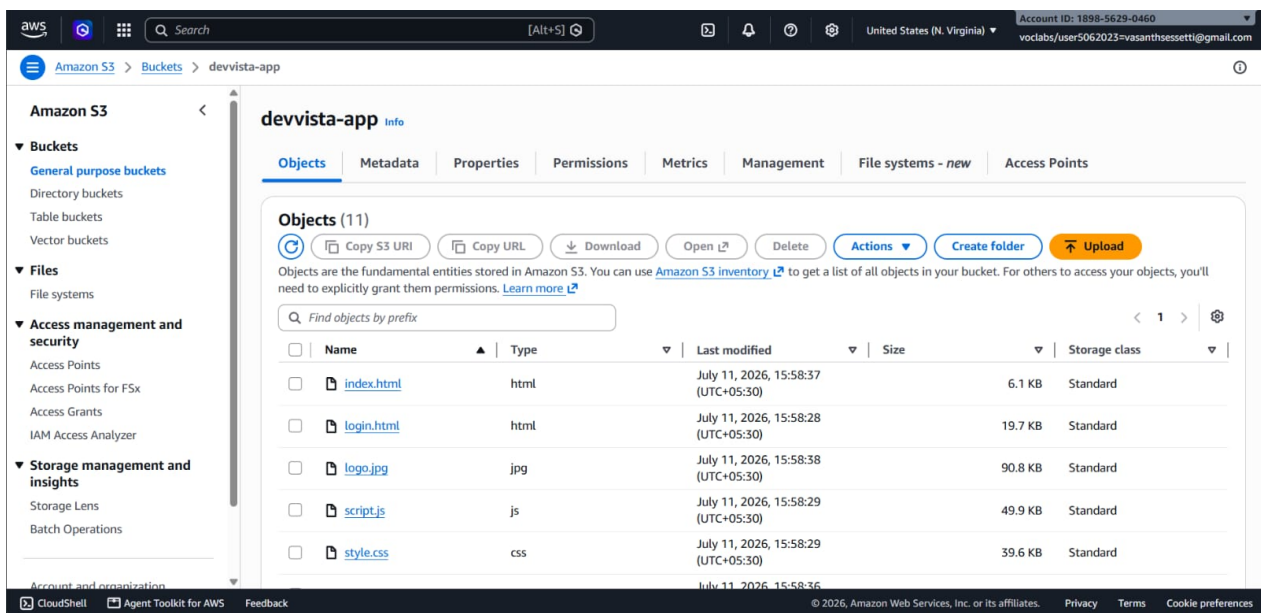
The User Signup API is responsible for handling new user registration requests in the DevVista Portfolio Generator. When a user submits the registration form, the request is received through Amazon API Gateway and forwarded to AWS Lambda for processing. The Lambda function validates the input data, checks for existing user records, and securely stores the user's information in Amazon DynamoDB. After successful registration, the API returns a confirmation response to the user. This implementation ensures secure, reliable, and efficient user account creation while maintaining data integrity and application performance.

4.5. USER LOGIN API

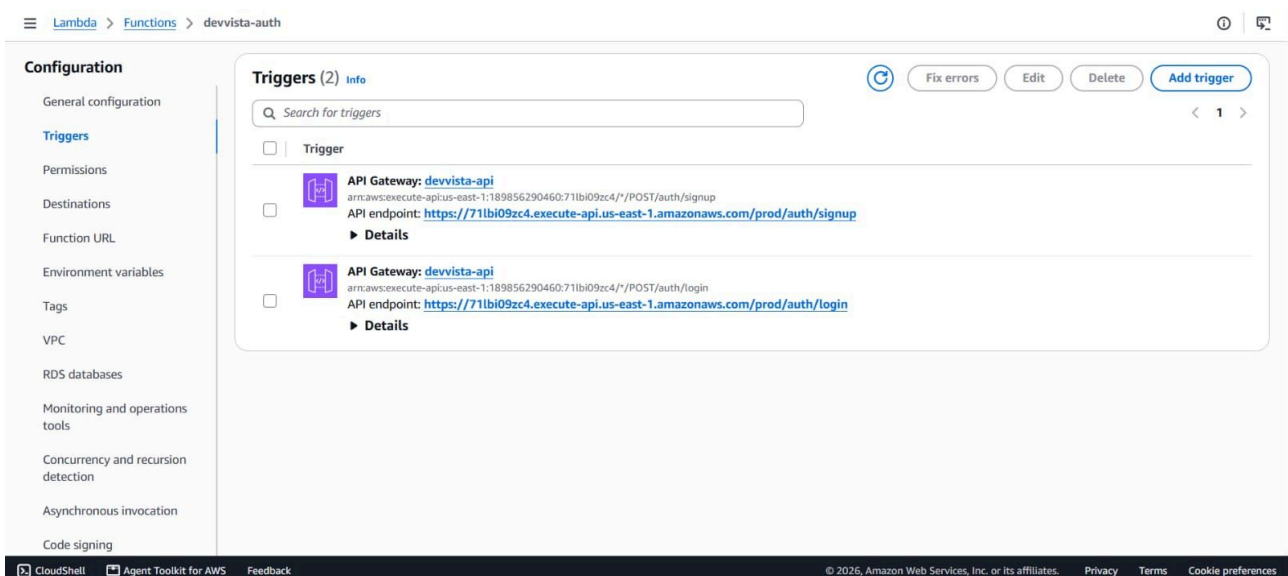
The User Login API is responsible for authenticating registered users in the DevVista Portfolio Generator. When a user enters their login credentials, the request is sent through Amazon API Gateway to AWS Lambda. The Lambda function verifies the provided email and password by checking the records stored in Amazon DynamoDB. If the credentials are valid, the user is successfully authenticated and granted access to the application. If the credentials are incorrect, the API returns an appropriate error message. This process ensures secure authentication, protects user data, and provides a reliable login experience.



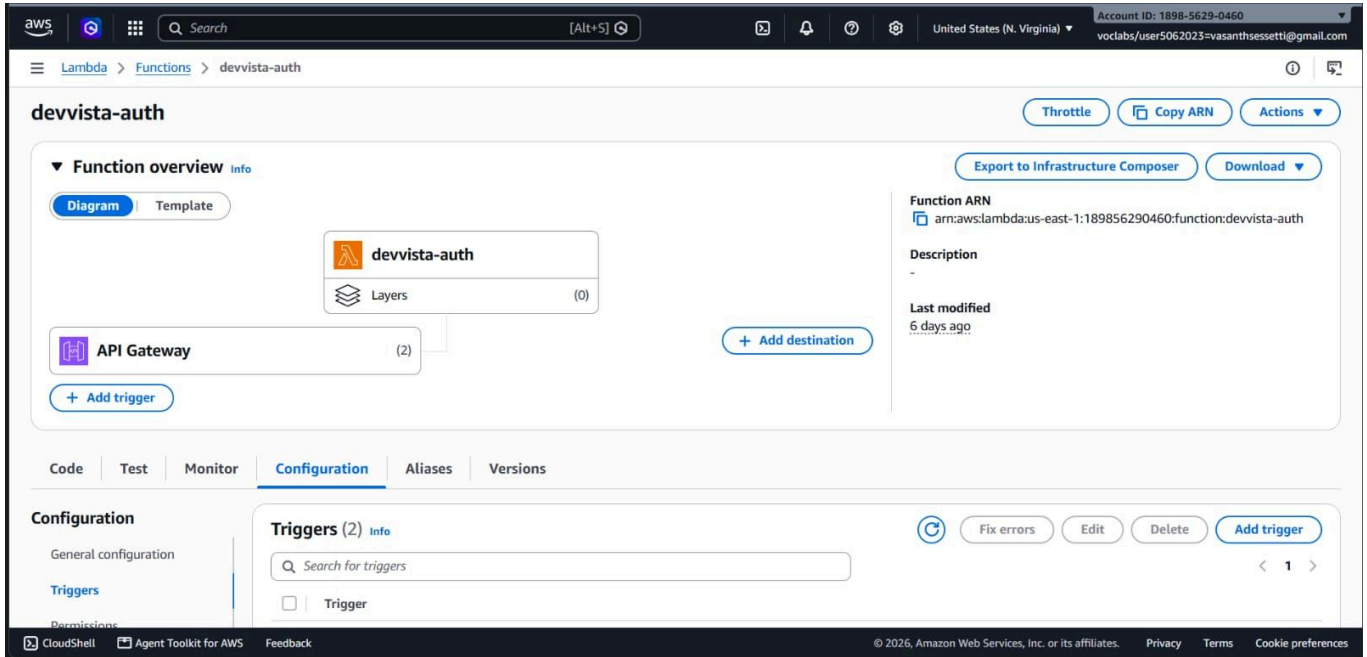
This S3 bucket is used to store all the files needed for the web application. It currently holds 11 files in total, including the main page (index.html), the login page (login.html), the application logo (logo.jpg), the functional script (script.js), and the design file (style.css). These files are stored using the Standard storage class to ensure the website runs correctly.



The devvista-auth AWS Lambda function acts as the core backend authentication service for the application. Its primary responsibility is to manage user identity and secure access by processing sign-up and login requests. By serving as the central authentication logic, it ensures that all user-related interactions are validated efficiently within the application infrastructure.

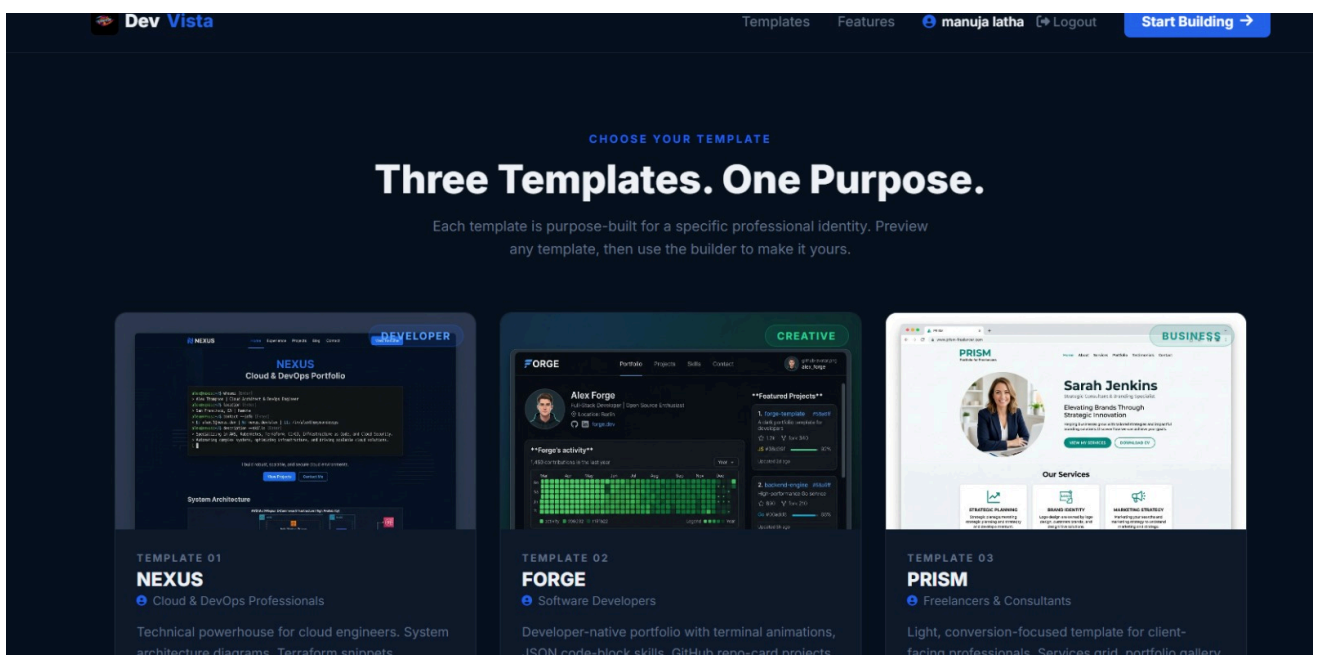
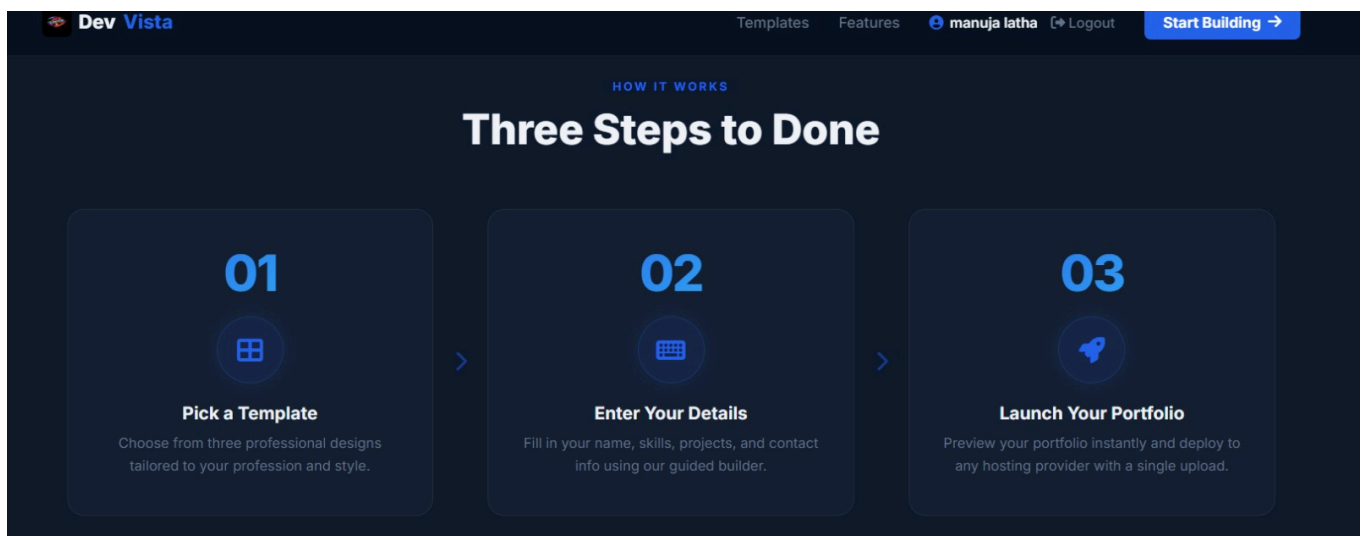
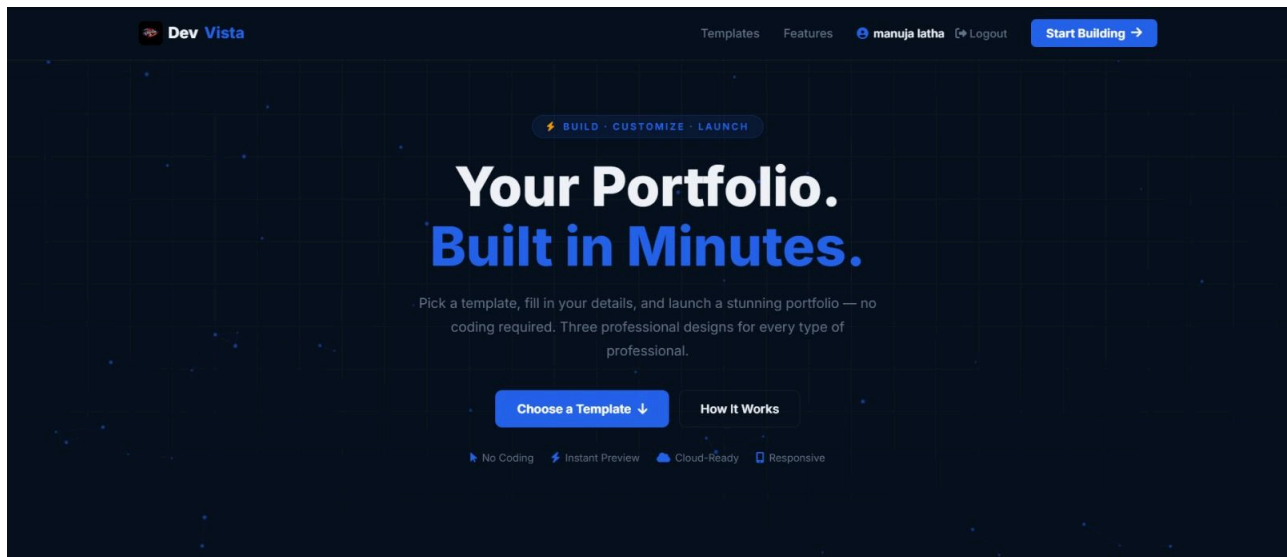


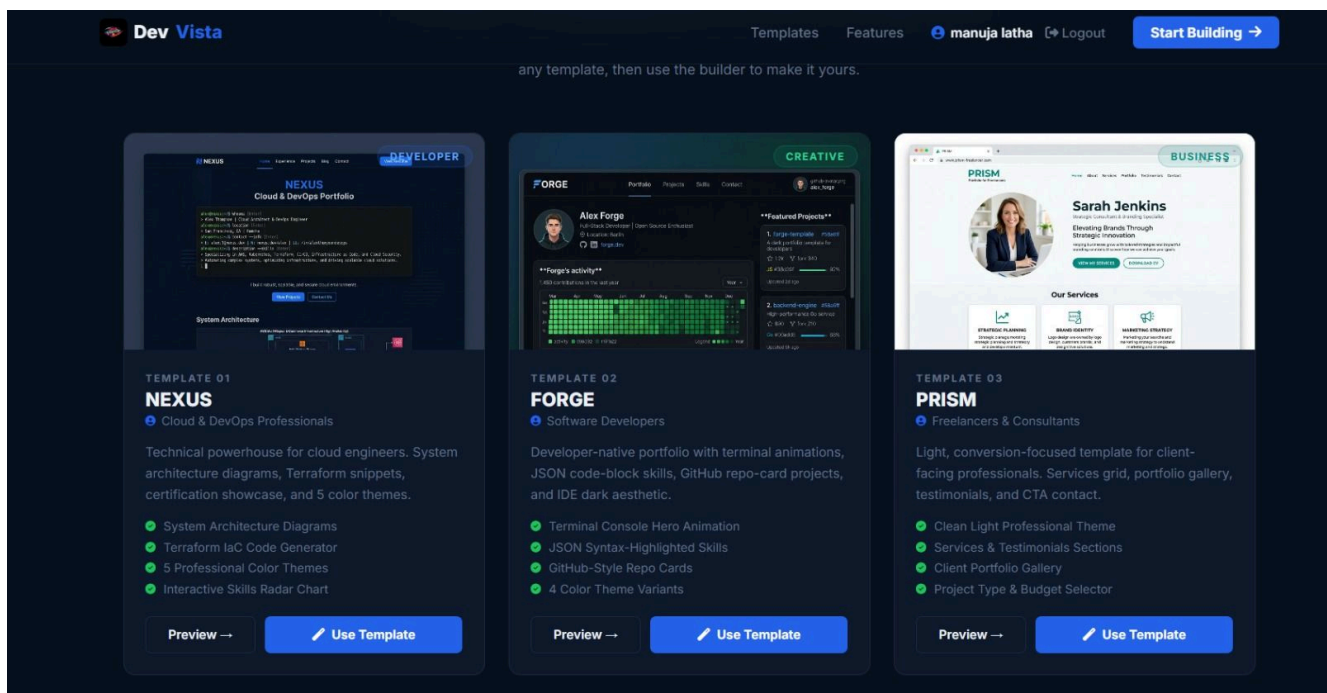
From an architectural perspective, the devvista-auth function is deployed within the us-east-1 region, ensuring reliable execution of its authentication tasks. The integration between the function and API Gateway creates a robust environment that bridges the client-side frontend with the backend security layer. This configuration is essential for maintaining a secure and functional user management system.



This function is tightly integrated with Amazon API Gateway, which provides the necessary endpoints to facilitate communication.

Specifically, it utilizes two distinct triggers under the devvista-api configuration: one dedicated to processing user sign-ups and another for handling login authentication. This setup allows the application to route incoming traffic seamlessly to the appropriate backend logic.





Dev Vista functions as an intuitive platform that allows professionals to build and launch portfolios in minutes without requiring coding knowledge. By offering three purpose-built templates—NEXUS for DevOps professionals, FORGE for software developers, and PRISM for freelancers—the platform caters to specific career identities. Users can customize these templates through a guided builder to reflect their unique skills, projects, and professional background.

The portfolio creation process is streamlined into three simple steps: picking a template, entering personal details like bio and contact information, and launching the final product. The builder includes a comprehensive profile configuration section and provides an instant preview of the portfolio layout. This final output highlights the user's "Open to Work" status, displays social media links, and provides quick access to resumes and project portfolios, creating a polished professional presence.

DevVista

Building with

FORGE

1. Profile

2. Contact

3. Skills

4. Experience

5. Education

6. Projects

7. Theme

8. Launch

Personal Profile

This info appears in your portfolio header. Fields marked * are required.

Profile Photo

Recommended: square image - JPG, PNG, WebP - Max 5 MB

Upload Photo

Upload Photo

FULL NAME *

PROFESSIONAL TITLE *

e.g. Alex Johnson

e.g. Cloud Architect & Developer

SHORT BIO

A brief, impactful professional summary...

LOCATION

PERSONAL WEBSITE

e.g. New York, USA

https://yourwebsite.com

Step 1 of 8

Next →

DevVista

Skills

Projects

Experience

Contact

Theme

Hire Me

← Studio

AVAILABLE FOR WORK

Your Name

Software Developer

Crafting elegant digital experiences with clean code and creative problem-solving. Passionate about building products that make a difference.

GitHub

in LinkedIn

X Twitter

Email

View My Work

Resume

Your Name

Software Developer

Open to Work

EXPERIENCE

PROJECTS

SKILLS

22 | Page

5. IMPLEMENTATION

The implementation of DevVista is carried out using a fully serverless architecture on Amazon Web Services. The main objective is to provide a platform where users can create and host professional portfolios without managing any infrastructure.

The frontend of the application is developed using HTML, CSS and JavaScript. All static files including ``index.html``, ``login.html``, ``script.js``, ``style.css``, and ``logo.jpg`` are stored in an Amazon S3 bucket named ``devvista-app``. The bucket has static website hosting enabled. Three responsive portfolio templates are created and stored in separate folders: ``t1/`` for NEXUS, ``t2/`` for FORGE, and ``t3/`` for PRISM. NEXUS is designed for Cloud and DevOps engineers with architecture diagrams, FORGE for Software Developers with GitHub integration style, and PRISM for Freelancers with a clean business layout.

For backend logic, AWS Lambda is used. A Lambda function named ``devvista-auth`` is created using Python 3.12 runtime. This function handles two main operations: user registration and user authentication. The function receives JSON data from the frontend, validates the input fields, hashes the password using the `bcrypt` library, and interacts with the database.

Amazon DynamoDB is used to store user information. A table named `devvista-users` is created with `email` as the partition key. Attributes stored are `name`, `passwordHash`, and `createdAt`. DynamoDB provides automatic scaling and low-latency access, which is ideal for a serverless application.

Amazon API Gateway is configured to expose two REST endpoints: `POST /auth/signup` and `POST /auth/login`. These endpoints are integrated with the `devvista-auth` Lambda function. CORS is enabled to allow communication between the S3 hosted frontend and API Gateway.

AWS IAM roles are created following the principle of least privilege. The Lambda execution role has permissions only to read and write to the `devvista-users` DynamoDB table. Amazon CloudWatch is enabled to capture logs of every Lambda invocation. This helps in debugging and monitoring the performance of the application.

The implementation of DevVista is carried out using a fully serverless architecture on Amazon Web Services. The main objective is to provide a platform where users can create and host professional portfolios without managing any infrastru

The S3 bucket successfully hosted the website and all three templates loaded without any delay. The navigation between landing page, login page and template preview worked correctly. User signup and login forms were tested. On submitting the signup form, the data was sent to API Gateway and processed by the Lambda function. New user records were successfully created in the DynamoDB table `devvista-users` with password stored in hashed format `\$2b\$10\$`. Login was also tested and validated against stored credentials.

The API Gateway responded to requests within 200-400ms. Multiple concurrent signup requests were sent to test scalability and Lambda automatically scaled to handle the load without any errors. Since all services are serverless, there was no downtime observed during testing.

IAM policies were verified to ensure Lambda could only access required DynamoDB actions. Passwords were not stored in plain text. CloudWatch logs confirmed that no sensitive data was logged during execution.

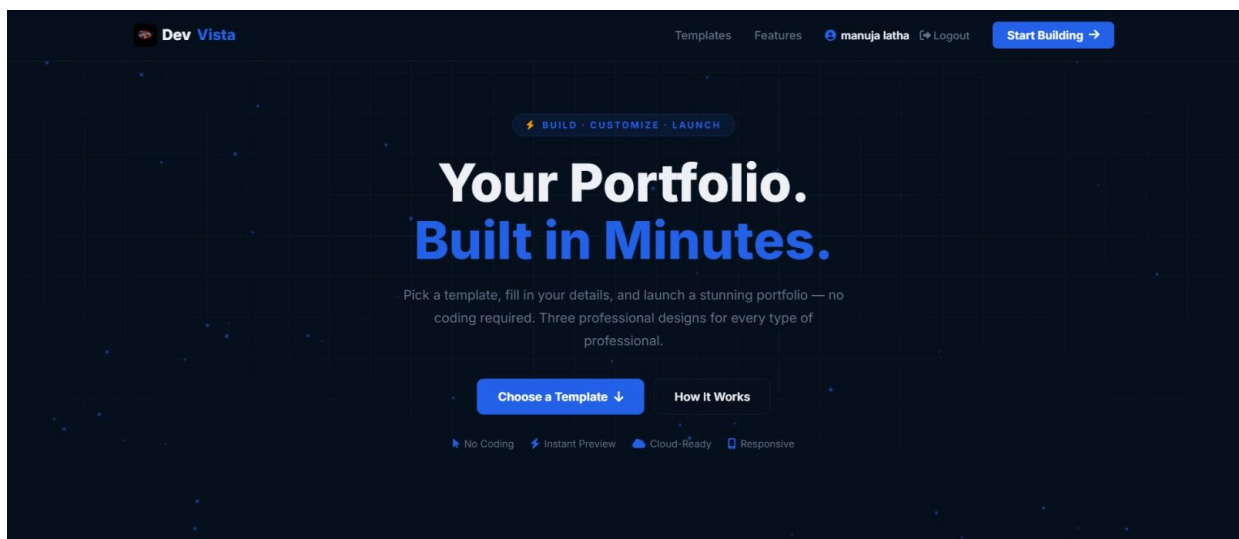
6. RESULTS

The DevVista application was successfully implemented and tested using AWS cloud services. The website was hosted on Amazon S3 and all pages loaded correctly without any issues. Users were able to register and log in successfully through the API Gateway and AWS Lambda integration.

The user details were stored securely in the Amazon DynamoDB table, with passwords saved in encrypted (hashed) format instead of plain text. API requests were processed efficiently, and the serverless architecture automatically handled user requests without requiring manual server management.

CloudWatch logs confirmed that all Lambda functions executed successfully, and no major runtime errors were observed during testing. The application demonstrated good performance, scalability, and security throughout the testing phase.

Overall, DevVista achieved its objectives by providing a reliable, secure, and user-friendly platform for creating and accessing professional portfolio templates. The successful integration of Amazon S3, API Gateway, AWS Lambda, DynamoDB, IAM, and CloudWatch proves the effectiveness of a serverless architecture for modern web applications.



7. CONCLUSION

The DevVista project successfully demonstrates the development of a modern serverless portfolio platform using Amazon WebServices (AWS).

By integrating Amazon S3, API Gateway, AW Lambda, and Amazon

DynamoDB, the application provides a secure, scalable, and cost-effective solution for hosting and managing user portfolios.

The serverless architecture eliminates the need for managing physical servers, reducing infrastructure costs while improving application reliability and scalability. Features such as secure user authentication, encrypted password storage, and responsive portfolio templates enhance the overall user experience.

The project also highlights the benefits of cloud computing, including high availability, automatic scaling, and simplified maintenance. AWS CloudWatch and IAM services further improve the application's monitoring and security.

In the future, DevVista can be enhanced by adding features such as custom portfolio editing, custom domain support, email verification, resume generation, portfolio analytics, AI-powered content suggestions, and social media integration.

In conclusion, DevVista is an efficient, secure, and scalable cloud-based application that enables users to create and showcase professional portfolios with ease. The project demonstrates the practical implementation of serverless cloud technologies for real-world web application development

Security was an important consideration throughout the development process. Passwords are securely encrypted before storage, IAM roles provide controlled access to AWS resources, and CloudWatch continuously monitors application activities. These security measures help protect user information and improve the reliability of the platform.

Another key achievement of the project is the availability of multiple responsive portfolio templates designed for different types of users, including software developers, cloud engineers, and freelancers. The simple interface and responsive design make the platform easy to use on both desktop and mobile devices, providing a better user experience.

The project demonstrates how cloud computing technologies can be effectively used to build modern web applications. It also provides practical experience in implementing serverless architecture, REST APIs, cloud storage, authentication, and database management using AWS services.